The anatomy of observability

At its heart, observability is about making sense of what's happening inside a system by analysing output data. This concept originated from control theory. It became very important in software engineering as systems became more distributed and dynamic, and it became challenging to manage them with conventional methods.

The three primary forms of telemetry data that build observability include logs, metrics and traces. The combined data enables teams to monitor their systems, detect issues and analyse behaviours during unexpected problems.

The system generates logs that maintain detailed records of its operational events. These logs contain error messages, API calls or records of user login attempts. The system health and performance trends become visible to users through these logs. Open source tools like Fluentd, Logstash and Grafana Loki serve as popular solutions for log collection, processing, and search functionality across multiple services and environments.

Metrics are numerical values that measure performance over time. Common system performance metrics include CPU usage, request latency and active user sessions. Prometheus stands as the leading open source platform to retrieve and analyse metrics and is paired with Grafana for interactive dashboard development.

Traces track the path of a request as it moves through different parts of an application. Microservices-based systems benefit from tracing because it helps developers to identify slowdowns and failures across multiple services. This is especially helpful when one user action may touch several services. Jaeger and Zipkin are leading open source tools that deliver robust distributed tracing solutions to users.

Each of these components tells a part of the story. But when used together, they form a complete picture (Table 1).

Table 1: Core observability data types

Data type	Description	Tools
Logs	Detailed event records	Fluentd, Logstash, Grafana Loki
Metrics	Numerical performance data	Prometheus
Traces	Request flow tracking	Jaeger, Zipkin

Observability vs monitoring: What's the difference?

The terms monitoring and observability are often confused, but they are distinct concepts. Both maintain system health but operate differently because they serve different needs in complex modern applications.

The main goal of monitoring is tracking known problems. You establish tracking parameters such as CPU usage, memory and error rates, and configure alert systems to trigger when thresholds are exceeded. The system functions effectively when you understand what constitutes a failure. A monitoring system such as Nagios, Zabbix or Prometheus (also used for observability) will alert you when a server crashes or CPU usage exceeds a certain threshold. Monitoring systems identify system problems, but they do not provide explanations about their causes.

Observability goes deeper. The system provides the necessary tools to investigate unexplained issues. The combination of logs, metrics and traces helps to understand the internal behaviour of the system, especially during unexpected failures. Tools such as Grafana, Jaeger, Loki and OpenTelemetry enable users to track how individual user requests move between multiple services. Observability provides complete visibility into system operations beyond individual metrics or alert notifications.

Think of monitoring like a car's dashboard -- it shows speed, fuel in the car, and gives engine warnings. Observability is more like having access to a mechanic's full diagnostic toolkit. The diagnostic equipment of a mechanic provides a far more comprehensive view than what monitoring tools offer. The system enables you to detect hidden problems, which you would not have looked for otherwise.

Monitoring functions as an alert system, which notifies users about system problems. Observability functions as an investigative tool that reveals the root causes of system failures and their initial points of origin.

Both are essential. System stability depends on monitoring, while observability gives an insight and helps improve the capabilities of systems during fast scaling or complex debugging situations. Open source tools offer cost-effective monitoring and observability solutions that can also be customised.

The open source observability landscape

Open source tools function as the fundamental building blocks for modern observability stacks. They are powerful, cost-effective and backed by strong communities. Most DevOps teams have adopted open source tools as their primary solution for application monitoring and debugging, as well as performance enhancement.

Prometheus is one of the leading tools. It is well known for metrics collection and querying, and performs best in Kubernetes environments. Through its real-time alert functionality Prometheus enables teams to quickly identify and manage system health problems.

Grafana uses Prometheus data to generate interactive dashboards that make complex data more understandable. The